Projet « Moteur de recherche » — Partie 2 sur 3

Analyse du contenu textuel et implémentation du moteur de recherche

Programmation de spécialité (Python) (≈5–6 h)

Julien Velcin* Walid Bechkit†

Année universitaire 2026–2027

Cette **deuxième partie** prolonge directement la Partie 1 : le corpus est désormais structuré en classes (`Document`, `Author`, `Corpus`). Nous allons maintenant *analyser le contenu textuel* des documents (expressions régulières, statistiques, construction du vocabulaire), puis *implémenter* un véritable **moteur de recherche** fondé sur une matrice Documents × Termes (scores TF et TF–IDF) et une mesure de similarité.

Conformément au principe du projet, les librairies « prêtes à l'emploi » (`scikit-learn`, `nltk`) restent **interdites** pour le cœur du moteur : la matrice Documents × Termes doit être construite *manuellement*. En revanche, `pandas` et `scipy` (pour les matrices creuses) sont autorisés. Cette partie correspond à la version **v2** du rendu.

Partie 1 : Travail sur les expressions régulières

1.1 Nous allons commencer par utiliser la librairie `re` (*regular expression*). Ajoutez une fonction `search` à la classe `Corpus`, qui retourne les passages des documents contenant le mot-clé entré en paramètre. Pour cela, il vous faudra travailler sur une unique chaîne de caractères concaténant l'intégralité des textes (cf. Partie 1). L'idéal est de ne pas reconstruire cette chaîne à chaque appel de `search`, mais une seule fois, au moment du **premier** appel.

1.2 Ajoutez une fonction `concorde` à la classe `Corpus`, qui construit un *concordancier* pour une expression donnée. Il s'agit d'une légère modification de la fonction `search`. La taille du contexte est fixée par un paramètre d'entrée. Vous ferez appel à la librairie `re` et à la librairie `pandas` afin de stocker et de retourner les résultats dans un tableau de la forme suivante :

contexte gauche	motif trouvé	contexte droit
...oilà un exemple de	texte	trouvé au milieu d...
...euxième exemple de	texte	trouvé ailleurs ma...

*Auteur du sujet original.

†Responsable du cours

Partie 2 : Quelques statistiques

Nous allons maintenant implémenter une méthode affichant plusieurs statistiques textuelles sur le corpus (que l'on appellera *stats*). Elle doit afficher :

- le nombre de mots **différents** dans le corpus ;
- les **n** mots les plus fréquents (`n` est un paramètre).

Pour cela, suivez les étapes ci-dessous.

2.1 Implémentez d'abord une fonction `nettoyer_texte` qui prend une chaîne de caractères en entrée et lui applique une série de traitements. Vous implémenterez au minimum : la mise en minuscules (via `lower()`) et le remplacement des passages à la ligne (`\n`). Vous pouvez aussi remplacer la ponctuation et les chiffres à l'aide d'expressions régulières appropriées.

2.2 En bouclant sur les documents du corpus, construisez **vous-même** le *vocabulaire* servant à décrire les textes. Vous utiliserez la fonction `split` en considérant plusieurs délimiteurs possibles pour l'anglais (espace, tabulation, signe de ponctuation...). Le vocabulaire sera stocké dans un dictionnaire ; vous pouvez passer au préalable par un ensemble (`set`) pour éliminer facilement les doublons.

2.3 Comptez enfin le nombre d'occurrences de chaque mot du vocabulaire en parcourant la liste des documents. Parcourir deux fois le corpus n'est évidemment pas le plus efficace : n'hésitez pas à chercher une solution qui évite cette perte de temps. L'idéal est de construire un tableau `freq` à l'aide de la librairie `pandas`.

2.4 Enrichissez le tableau `freq` en ajoutant une colonne indiquant le nombre de **documents** contenant chaque mot (*document frequency*). Attention : ce n'est pas la même chose que le nombre total d'occurrences du mot dans le corpus (*term frequency*).

Partie 3 : Matrice Documents $\times$ Mots

Le moteur que nous allons construire se fonde uniquement sur la présence de mots-clés correspondant à une requête de l'utilisateur. Il repose sur la construction **manuelle** d'une matrice Documents $\times$ Termes. Rappel : cette matrice ne doit pas être construite automatiquement par une librairie externe comme `scikit-learn`.

3.1 Construisez d'abord un dictionnaire contenant les mots servant à décrire le texte de vos documents. Vous pouvez réutiliser le découpage en mots de la partie précédente, sans oublier de retirer les doublons et de trier par ordre alphabétique. Vous appellerez ce dictionnaire **vocab** : les clés sont les mots eux-mêmes, et la valeur est un *nouveau* dictionnaire contenant plusieurs informations sur le mot (son identifiant unique, son nombre total d'occurrences...).

3.2 Parcourez ensuite votre collection de documents afin de construire une matrice de dimension (nombre de documents) × (nombre de mots du vocabulaire). À l'intersection de la ligne i (document) et de la colonne j (mot), vous placerez le nombre d'occurrences du mot dans le texte (c'est-à-dire le *Term Frequency*, ou TF). La matrice étant très creuse, l'idéal est d'utiliser une classe spécialement conçue pour cela, comme `sparse.csr_matrix` de la bibliothèque `scipy`. Vous appellerez cette matrice **mat_TF**.

3.3 À partir de cette matrice, calculez pour chaque mot le nombre total d'occurrences dans le corpus et le nombre total de documents le contenant. Vous stockerez ces informations dans **vocab**.

3.4 Une alternative intéressante au score TF est la mesure TF–IDF (cf. https://fr.wikipedia.org/wiki/TF-IDF). Calculez une seconde matrice, appelée **mat_TFxIDF**, qui implémente cette mesure.

Partie 4 : Moteur de recherche

Pour réaliser votre moteur de recherche, les principales étapes sont les suivantes :

- demander à l'utilisateur d'entrer quelques mots-clés ;
- transformer ces mots-clés en un **vecteur** sur le vocabulaire précédemment construit ;
- calculer une **similarité** entre le vecteur requête et chaque document ;
- trier les scores et afficher les meilleurs résultats.

La similarité peut être calculée par un simple **produit scalaire** entre le vecteur requête et le vecteur du document visé. Une mesure souvent plus appropriée est celle du **cosinus** (cf. https://fr.wikipedia.org/wiki/Similarite_cosinus).

Partie 5 : Intégrer le tout dans une classe `SearchEngine`

Pour finir, intégrez l'ensemble du code produit dans cette partie au sein d'une classe `SearchEngine`. Cette classe doit respecter les contraintes suivantes :

- on doit pouvoir lui fournir un objet de type `Corpus` lors de l'instanciation : la construction de la matrice Documents × Termes se fait alors dans la foulée ;
- elle doit proposer une méthode `search` prenant deux arguments : les mots-clés de la requête et le nombre de documents à retourner à l'utilisateur ;
- le résultat de la recherche doit être retourné sous la forme d'une table `DataFrame` de `pandas`.